

1. Explain with example Relations, tuples and Filtering in context of Pig.

1. Tuples

A **tuple** is an ordered set of fields, where a field is a piece of data. It is similar to a row in a relational database table. [\[1, 2\]](#)

- **Structure:** Represented by parentheses ().
- **Characteristics:** Fields within a tuple can be of any data type (int, chararray, etc.). A tuple does not require all tuples in a relation to have the same number of fields.
- **Example:** ('John', 25, 'Engineering') [\[1, 2, 3, 4, 5\]](#)

2. Relations

A **relation** is the outermost structure, technically a "bag" (an unordered collection) of tuples. It is similar to a table in a RDBMS, but Pig relations are generally unordered, meaning there is no guarantee that tuples are processed in any specific order. [\[1, 2\]](#)

- **Structure:** Relations are assigned names (aliases) in Pig script.

Example:

pig

-- Loading a file into a relation 'Students'

Students = LOAD 'student_data.txt' AS (name:chararray, age:int, city:chararray);

If `student_data.txt` contains:

Alice,20,New York

Bob,22,Chennai

Charlie,21,Chennai

- The relation `Students` acts as a container for these three tuples. [\[1, 2, 3, 4, 5\]](#)

3. Filtering

The **FILTER** operator is used to select specific tuples from a relation based on a specified condition (predicate). It is equivalent to the `WHERE` clause in SQL. [\[1, 2, 3\]](#)

- **Syntax:** `alias = FILTER relation_name BY condition;`
- **Example:** Based on the `Students` relation above, let's filter for students living in 'Chennai'. [\[1\]](#)

pig

```
-- Filtering the relation
chennai_students = FILTER Students BY city == 'Chennai';

-- Viewing the output
DUMP chennai_students;
```

Output:

```
(Bob, 22, Chennai)
(Charlie, 21, Chennai)
```

Comprehensive Example

Imagine a file named `employee.txt` in HDFS containing employee data:

text

```
101,John,Sales,5000
102,Doe,IT,7000
103,Smith,Sales,4000
104,Jane,IT,9000
```

Here is the Pig script illustrating the concepts:

pig

```
-- 1. RELATION: Loading data and creating a relation 'emp'
```

```
emp = LOAD 'employee.txt' USING PigStorage(',') AS (id:int, name:chararray, dept:chararray, salary:int);
```

```
-- 2. FILTERING: Selecting tuples where dept is 'Sales'  
sales_emp = FILTER emp BY dept == 'Sales';
```

```
-- 3. FILTERING: Selecting tuples where salary > 6000  
high_salary = FILTER emp BY salary > 6000;
```

```
-- TUPLES: The output of DUMP will show the filtered tuples  
DUMP sales_emp;
```

```
-- Output: (101,John,Sales,5000), (103,Smith,Sales,4000)
```

```
DUMP high_salary;
```

```
-- Output: (102,Doe,IT,7000), (104,Jane,IT,9000)
```

2. Write and describe various Pig relational operators.

1. Input/Output Operators

These operators load data into Pig and store the processed results. [\[1, 2, 3\]](#)

- **LOAD**: Loads data from the file system (HDFS/Local) into a relation. It can define schema and specify loaders (e.g., `PigStorage`, `JsonLoader`).
 - *Example*: `A = LOAD 'data.txt' USING PigStorage(',') AS (name:chararray, age:int);`
- **STORE**: Saves the results of a relation to a file system.
 - *Example*: `STORE A INTO 'output_dir' USING PigStorage('\t');`
- **DUMP**: Displays the contents of a relation directly to the screen (useful for debugging).
 - *Example*: `DUMP A;` [\[1, 2, 3, 4, 5\]](#)

2. Filtering and Transformation Operators

These operators modify, select, or remove data from a relation. [\[1, 2, 3\]](#)

- **FILTER**: Selects rows based on a specific condition.
 - *Example*: `B = FILTER A BY age > 20;`
- **FOREACH ... GENERATE**: Processes rows to generate data transformations (similar to SELECT in SQL), allowing for field selection or projections.

- *Example:* `C = FOREACH A GENERATE name, age * 12;`
- **LIMIT:** Restricts the number of output tuples.
 - *Example:* `D = LIMIT A 10;`
- **DISTINCT:** Removes duplicate tuples from a relation.
 - *Example:* `B = DISTINCT A;`
- **STREAM:** Allows using external scripts (Python, Perl) to process data.
 - *Example:* `B = STREAM A THROUGH my_script.py;` [[1](#), [2](#), [3](#), [4](#), [5](#)]

3. Grouping and Joining Operators

These operators are used to merge or organize data from one or more relations based on keys. [[1](#), [2](#)]

- **GROUP:** Groups data in a single relation based on a key. It produces a nested structure.
 - *Example:* `B = GROUP A BY name;`
- **COGROUP:** Similar to GROUP, but works on two or more relations simultaneously.
 - *Example:* `C = COGROUP A BY name, B BY name;`
- **JOIN (Inner/Outer):** Joins two or more relations based on common fields.
 - *Example:* `C = JOIN A BY name, B BY name;`
- **CROSS:** Computes the Cartesian product of two or more relations.
 - *Example:* `C = CROSS A, B;` [[1](#), [2](#), [3](#), [4](#), [5](#)]

4. Sorting and Combining Operators

These operators rearrange or merge multiple relations. [[1](#), [2](#)]

- **ORDER BY:** Sorts a relation based on one or more fields.
 - *Example:* `B = ORDER A BY age DESC;`
- **UNION:** Merges the contents of two or more relations (concatenation).
 - *Example:* `C = UNION A, B;`
- **SPLIT:** Divides a single relation into two or more relations based on conditions.
 - *Example:* `SPLIT A INTO A1 IF age < 20, A2 IF age >= 20;` [[1](#), [2](#), [3](#), [4](#), [5](#)]

5. Diagnostic Operators

These operators are used to inspect the schema or structure of data during development.

- **DESCRIBE**: Shows the schema of a relation (field names and types).
 - *Example*: `DESCRIBE A;`
- **EXPLAIN**: Displays the logical, physical, or MapReduce execution plan for a relation.
 - *Example*: `EXPLAIN A;`
- **ILLUSTRATE**: Shows step-by-step data transformations for a sample of data.
 - *Example*: `ILLUSTRATE A;` [[1](#), [2](#), [3](#), [4](#), [5](#)]

Summary Table of Common Operators

Operator [1 , 2 , 3 , 4 , 5]	Type	Description
LOAD	I/O	Reads data from HDFS.
STORE	I/O	Saves data to HDFS.
FILTER	Transform	Selects rows satisfying a condition.
FOREACH	Transform	Applies expressions to rows.
GROUP	Grouping	Groups data by key.
JOIN	Joining	Joins relations.

ORDER	Sorting	Sorts data.
--------------	---------	-------------

DISTINCT	Transform	Removes duplicates.
-----------------	-----------	---------------------

UNION	Combining	Merges relations.
--------------	-----------	-------------------

SPLIT	Combining	Splits relations.
--------------	-----------	-------------------